

```
// =====
// SCHEMA ADDITIONS FOR SCALABLE ARCHITECTURE
// =====
// Add these models to prisma/schema.prisma
// Run: yarn prisma db push

// =====
// LEADERBOARD SNAPSHOTS (CQRS Read Model)
// =====
// Purpose: Pre-computed leaderboard rankings to avoid
// expensive real-time aggregations

model LeaderboardSnapshot {
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Snapshot identification
  periodType String // 'weekly' | 'monthly' | 'all_time'
  periodKey String // '2026-W09' for weekly, '2026-03' for monthly, 'all_time'

  // Aggregated metrics (denormalized for fast reads)
  totalPoints Int @default(0)
  streakDays Int @default(0)
  challengesCompleted Int @default(0)
  mealsLogged Int @default(0)
  aiInteractions Int @default(0)
  referralsConverted Int @default(0)

  // Computed ranking
  rank Int?
  percentile Float? // e.g., 95.5 means top 4.5%

  // Metadata
  computedAt DateTime @default(now())
  expiresAt DateTime?

  @@unique([userId, periodType, periodKey])
  @@index([periodType, periodKey, totalPoints(sort: Desc)])
  @@index([periodType, periodKey, rank])
  @@index([computedAt])
  @@index([userId])
}

// =====
// JOB QUEUE (Persistent job tracking)
// =====
// Purpose: Track async jobs with retry logic
// Note: BullMQ handles the actual queue, this is for persistence/audit

model JobQueue {
  id String @id @default(cuid())
```

```

// Job identification
jobType String // 'PLAN_GENERATION' | 'SHARE_CARD' | 'EMBEDDING' | 'LEADERBOARD_SNAPSHOT' |
'EMAIL'
externalJobId String? // BullMQ job ID for correlation

// Ownership
userId String?
user User? @relation(fields: [userId], references: [id], onDelete: SetNull)

// Payload
payload Json // Job-specific data

// Status tracking
status String @default("pending") // pending | processing | completed | failed | cancelled
priority Int @default(0) // Higher = more priority
attempts Int @default(0)
maxAttempts Int @default(3)

// Scheduling
scheduledFor DateTime @default(now())
startedAt DateTime?
completedAt DateTime?

// Results
result Json? // Success result
error String? // Error message
errorDetails Json? // Full error context

// Timestamps
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

@@index([status, scheduledFor])
@@index([jobType, status])
@@index([userId, jobType])
@@index([externalJobId])
}

// =====
// ANALYTICS EVENTS (High-volume event store)
// =====
// Purpose: Store all analytics events for analysis
// Note: Consider table partitioning by eventMonth in raw SQL

model AnalyticsEvent {
  id String @id @default(cuid())

  // User context (nullable for anonymous)
  userId String?
  user User? @relation(fields: [userId], references: [id], onDelete: SetNull)
  sessionId String // Browser session ID

```

```

// Event data
eventType String // PAGE_VIEW | CALCULATOR_USE | SIGNUP | CHALLENGE_JOINED | etc.
eventData Json? // Event-specific payload

// Attribution
source String? // seo | social | referral | direct | product_hunt
medium String? // organic | paid | email
campaign String? // UTM campaign
referrer String? // Full referrer URL
landingPage String? // First page in session

// Context
userAgent String?
ipHash String? // SHA256 hash for privacy
country String? // GeoIP derived
device String? // mobile | tablet | desktop

// Partitioning key (for table partitioning)
eventMonth String // '2026-03' format

// Conversion tracking
converted Boolean @default(false)
conversionId String? // Links to signup/purchase

// Timestamps
createdAt DateTime @default(now())

@@index([eventMonth, eventType])
@@index([userId, createdAt])
@@index([sessionId, createdAt])
@@index([source, eventMonth])
@@index([converted, eventMonth])
}

// =====
// SHARE CARDS (OG Image metadata)
// =====
// Purpose: Track generated share card images in S3

model ShareCard {
  id String @id @default(cuid())

  // Link to share
  shareId String @unique
  shareType String // 'challenge' | 'profile' | 'achievement' | 'meal_plan'

  // S3 storage reference
  s3Key String // Full S3 key path
  s3Bucket String // Bucket name
  cdnUrl String // CloudFront URL

  // Image metadata
  width Int @default(1200)
  height Int @default(630)

```

```

format String @default("png") // png | webp
sizeBytes Int?

// Template info
template String @default("default") // Template used
templateData Json? // Data passed to template

// Cache management
generatedAt DateTime @default(now())
expiresAt DateTime?
invalidatedAt DateTime?

// Generation tracking
generationTimeMs Int? // How long it took to generate

@@index([shareId])
@@index([shareType, generatedAt])
@@index([expiresAt])
}

// =====
// RATE LIMIT LOG (Backup to Redis)
// =====
// Purpose: Persistent rate limit tracking for analytics
// Primary rate limiting is in Redis, this is for audit

model RateLimitLog {
  id String @id @default(cuid())

  // Identifier
  identifier String // userId or IP hash
  identifierType String // 'user' | 'ip' | 'api_key'

  // Endpoint
  endpoint String // API path pattern
  method String // GET | POST | etc.

  // Window tracking
  windowStart DateTime
  windowDuration Int // seconds

  // Counts
  requestCount Int @default(1)
  limitExceeded Boolean @default(false)

  // Context
  userAgent String?

  createdAt DateTime @default(now())

  @@unique([identifier, endpoint, windowStart])
  @@index([windowStart])
  @@index([identifier, limitExceeded])
}

```

```
// =====
// SYSTEM METRICS (Internal monitoring)
// =====
// Purpose: Store system health metrics for dashboards

model SystemMetric {
  id String @id @default(cuid())

  // Metric identification
  metricName String // e.g., 'api_latency_p95', 'queue_depth', 'cache_hit_rate'
  metricType String // gauge | counter | histogram

  // Value
  value Float
  unit String? // ms | count | percent | bytes

  // Dimensions (for filtering/grouping)
  dimensions Json? // { "endpoint": "/api/chat", "region": "us-east-1" }

  // Time
  timestamp DateTime
  aggregationPeriod String? // '1m' | '5m' | '1h' | '1d'

  createdAt DateTime @default(now())

  @@index([metricName, timestamp])
  @@index([timestamp])
}

// =====
// FEATURE FLAGS (Runtime configuration)
// =====
// Purpose: Control feature rollout without deploys

model FeatureFlag {
  id String @id @default(cuid())

  // Flag identification
  name String @unique // e.g., 'new_meal_plan_ui', 'ai_coach_v2'
  description String?

  // Targeting
  enabled Boolean @default(false)
  enabledFor Json? // { "userIds": [], "percentRollout": 10 }

  // Metadata
  createdBy String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  expiresAt DateTime?

  @@index([name])
  @@index([enabled])
}
```

```
// =====
// REQUIRED RELATION ADDITIONS TO User MODEL
// =====
// Add these fields to the existing User model:

// model User {
// ... existing fields ...
//
// // Add these relations:
// leaderboardSnapshots LeaderboardSnapshot[]
// jobs JobQueue[]
// analyticsEvents AnalyticsEvent[]
// }

// =====
// RAW SQL FOR TABLE PARTITIONING
// =====
// Run this manually in production for high-volume event table:

/*
- Create partitioned analytics_events table
CREATE TABLE analytics_events_partitioned (
id TEXT PRIMARY KEY,
user_id TEXT,
session_id TEXT NOT NULL,
event_type TEXT NOT NULL,
event_data JSONB,
source TEXT,
medium TEXT,
campaign TEXT,
referrer TEXT,
landing_page TEXT,
user_agent TEXT,
ip_hash TEXT,
country TEXT,
device TEXT,
event_month TEXT NOT NULL,
converted BOOLEAN DEFAULT FALSE,
conversion_id TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY LIST (event_month);

- Create monthly partitions (automate via cron)
CREATE TABLE analytics_events_2026_03
PARTITION OF analytics_events_partitioned
FOR VALUES IN ('2026-03');

CREATE TABLE analytics_events_2026_04
PARTITION OF analytics_events_partitioned
FOR VALUES IN ('2026-04');
```

- Create indexes on partitioned table

```
CREATE INDEX idx_analytics_events_user ON analytics_events_partitioned (user_id, created_at);
CREATE INDEX idx_analytics_events_session ON analytics_events_partitioned (session_id, created_at);
CREATE INDEX idx_analytics_events_type ON analytics_events_partitioned (event_type, event_month);
CREATE INDEX idx_analytics_events_source ON analytics_events_partitioned (source, event_month);
```

- Automated partition creation (run monthly via pg\_cron or external scheduler)

```
CREATE OR REPLACE FUNCTION create_analytics_partition()
RETURNS void AS $$
DECLARE
next_month TEXT;
partition_name TEXT;
BEGIN
next_month := TO_CHAR(NOW() + INTERVAL '1 month', 'YYYY-MM');
partition_name := 'analytics_events_' || REPLACE(next_month, '-', '_');

EXECUTE format(
'CREATE TABLE IF NOT EXISTS %I PARTITION OF analytics_events_partitioned FOR VALUES IN (%L)',
partition_name,
next_month
);
END;
$$ LANGUAGE plpgsql;
```

- Drop old partitions (retention policy)

```
CREATE OR REPLACE FUNCTION cleanup_old_analytics_partitions(months_to_keep INT DEFAULT 12)
RETURNS void AS $$
DECLARE
cutoff_month TEXT;
partition_record RECORD;
BEGIN
cutoff_month := TO_CHAR(NOW() - (months_to_keep || ' months')::INTERVAL, 'YYYY-MM');

FOR partition_record IN
SELECT tablename FROM pg_tables
WHERE tablename LIKE 'analytics_events_20%'
AND tablename < 'analytics_events_' || REPLACE(cutoff_month, '-', '_')
LOOP
EXECUTE format('DROP TABLE IF EXISTS %I', partition_record.tablename);
END LOOP;
END;
$$ LANGUAGE plpgsql;
*/
```

```
// =====
// pgvector SETUP FOR RAG
// =====
```

```
/*
```

- Enable extension

```
CREATE EXTENSION IF NOT EXISTS vector;
```

– Embeddings table (if not using PdfChunk)

```
CREATE TABLE embeddings (
  id SERIAL PRIMARY KEY,
  chunk_id TEXT REFERENCES pdf_chunks(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  embedding vector(1536), -- OpenAI ada-002
  metadata JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

– HNSW index for fast approximate nearest neighbor search

```
CREATE INDEX ON embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

– Alternative: IVFFlat for larger datasets

```
CREATE INDEX ON embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

– Function for similarity search

```
CREATE OR REPLACE FUNCTION search_similar_chunks(
  query_embedding vector(1536),
  match_threshold FLOAT DEFAULT 0.7,
  match_count INT DEFAULT 5
)
RETURNS TABLE (
  chunk_id TEXT,
  content TEXT,
  similarity FLOAT
) AS $$
BEGIN
  RETURN QUERY
  SELECT
    e.chunk_id,
    e.content,
    1 - (e.embedding <=> query_embedding) as similarity
  FROM embeddings e
  WHERE 1 - (e.embedding <=> query_embedding) > match_threshold
  ORDER BY e.embedding <=> query_embedding
  LIMIT match_count;
END;
$$ LANGUAGE plpgsql;
*/
```